

Estructuras de Datos - Algoritmo de Floyd-Warshall

Introduccion

El algoritmo de Floyd-Warshall es una tecnica utilizada para encontrar los caminos mas cortos entre todos los pares de vertices en un grafo ponderado. Este algoritmo es especialmente util en problemas de redes, transporte y analisis de sistemas complejos.

Objetivo

Dado un grafo representado mediante una matriz de adyacencia, el objetivo es calcular la matriz de distancias mas cortas entre cada par de vertices i y j . Esto incluye considerar todos los posibles vertices intermedios para minimizar la distancia.

Idea General

1. Inicia con una matriz de distancias basada en las aristas directas del grafo. Si no existe una conexion directa entre dos vertices, el peso se considera infinito (∞).
2. Itera sobre cada vertice como intermedio k , verificando si el camino entre i y j se mejora al pasar por k .
3. Actualiza la matriz si se encuentra una distancia mas corta utilizando la formula:

$$D[i][j] = \min(D[i][j], D[i][k] + D[k][j]). \quad (1)$$

Construccion de la Matriz Inicial

La matriz inicial D se construye con base en las conexiones directas entre los vertices del grafo:

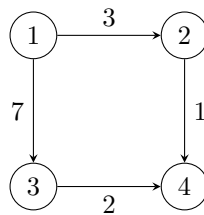
- Si existe una arista directa entre los vertices i y j , entonces $D[i][j]$ se establece como el peso de esa arista.
- Si no hay conexion directa, se asigna ∞ (infinito).
- La diagonal principal $D[i][i]$ siempre es 0 porque la distancia de un vertice a si mismo es 0.

Funcionamiento del Algoritmo

- Inicializacion:
 - Crea una matriz D donde $D[i][j]$ es igual al peso de la arista entre i y j , o ∞ si no hay conexion.
 - La diagonal principal $D[i][i]$ se inicializa en 0.
- Iteracion principal:
 - Para cada vertice intermedio k , actualiza las distancias considerando caminos que pasan por k .
- Resultado:
 - La matriz final D contiene las distancias mas cortas entre todos los pares de vertices.

Ejemplo Practico

Grafo inicial:



Paso 1: Representacion inicial del grafo en la matriz D .

$$D = \begin{bmatrix} 0 & 3 & 7 & \infty \\ \infty & 0 & \infty & 1 \\ \infty & \infty & 0 & 2 \\ \infty & \infty & \infty & 0 \end{bmatrix}$$

- La diagonal principal es 0 porque la distancia de un vertice a si mismo es siempre 0.
- Los valores ∞ indican que no hay conexion directa entre esos vertices.

Paso 2: Iteraciones del algoritmo.

1. Iteracion 1: Considerando el vertice 1 como intermedio.

- No se encuentran mejoras, ya que ningun camino pasando por el vertice 1 reduce las distancias actuales.

2. Iteracion 2: Considerando el vertice 2 como intermedio.

- La distancia entre los vertices 1 y 4 se actualiza:

$$D[1][4] = \min(D[1][4], D[1][2] + D[2][4]) = \min(\infty, 3 + 1) = 4$$

3. Iteracion 3: Considerando el vertice 3 como intermedio.

- No se encuentran mejoras adicionales.

4. Iteracion 4: Considerando el vertice 4 como intermedio.

- No se encuentran mejoras adicionales.

Matriz final:

$$D = \begin{bmatrix} 0 & 3 & 7 & 4 \\ \infty & 0 & \infty & 1 \\ \infty & \infty & 0 & 2 \\ \infty & \infty & \infty & 0 \end{bmatrix}$$

Codigo en Java

```
1 public class FloydWarshall {
2     public static final int INF = 99999; // Representa infinito
3
4     public void floydWarshall(int[][] graph) {
5         int n = graph.length;
6         int[][] dist = new int[n][n];
7
8         // Inicializar la matriz de distancias con los valores del grafo
9         for (int i = 0; i < n; i++) {
10             for (int j = 0; j < n; j++) {
11                 dist[i][j] = graph[i][j];
12             }
13         }
14
15         // Aplicar el algoritmo
16         for (int k = 0; k < n; k++) {
17             for (int i = 0; i < n; i++) {
18                 for (int j = 0; j < n; j++) {
19                     if (dist[i][k] + dist[k][j] < dist[i][j]) {
20                         dist[i][j] = dist[i][k] + dist[k][j];
21                     }
22                 }
23             }
24         }
25
26         // Imprimir la matriz final
27         printSolution(dist);
28     }
29
30     private void printSolution(int[][] dist) {
31         System.out.println("Matriz de distancias mas cortas:");
32         for (int i = 0; i < dist.length; i++) {
33             for (int j = 0; j < dist.length; j++) {
34                 if (dist[i][j] == INF) {
35                     System.out.print("INF ");
36                 } else {
37                     System.out.print(dist[i][j] + " ");
38                 }
39             }
40         }
41     }
42 }
```

```

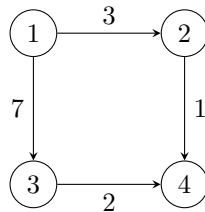
40         System.out.println();
41     }
42 }
43
44 public static void main(String[] args) {
45     int[][] graph = {
46         {0, 3, 7, INF},
47         {INF, 0, INF, 1},
48         {INF, INF, 0, 2},
49         {INF, INF, INF, 0}
50     };
51
52     FloydWarshall fw = new FloydWarshall();
53     fw.floydWarshall(graph);
54 }
55 }

```

Ejercicio 1: Facil

Enunciado: Dado el siguiente grafo dirigido y ponderado, encuentra las distancias mas cortas entre todos los pares de vertices utilizando el algoritmo de Floyd-Warshall.

Grafo:



Resolucion:

- Inicializa la matriz de distancias:

$$D = \begin{bmatrix} 0 & 3 & 7 & \infty \\ \infty & 0 & \infty & 1 \\ \infty & \infty & 0 & 2 \\ \infty & \infty & \infty & 0 \end{bmatrix}$$

- Iteraciones del algoritmo:
 - Considerando el vertice 1 como intermedio: No se producen cambios.
 - Considerando el vertice 2 como intermedio: Actualiza $D[1][4] = 4$.
 - Considerando el vertice 3 como intermedio: No se producen cambios.
 - Considerando el vertice 4 como intermedio: No se producen cambios.

- Resultado final:

$$D = \begin{bmatrix} 0 & 3 & 7 & 4 \\ \infty & 0 & \infty & 1 \\ \infty & \infty & 0 & 2 \\ \infty & \infty & \infty & 0 \end{bmatrix}$$

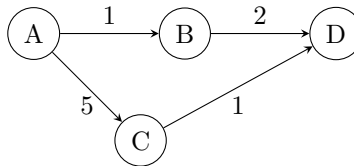
Codigo en Java:

```
1 int[][] graph = {  
2     {0, 3, 7, INF},  
3     {INF, 0, INF, 1},  
4     {INF, INF, 0, 2},  
5     {INF, INF, INF, 0}  
6 };  
7  
8 FloydWarshall fw = new FloydWarshall();  
9 fw.floydWarshall(graph);
```

Ejercicio 2: Intermedio

Enunciado: En el siguiente grafo dirigido y ponderado, encuentra las distancias mas cortas entre todos los pares de vertices.

Grafo:



Resolucion:

- Matriz inicial:

$$D = \begin{bmatrix} 0 & 1 & 5 & \infty \\ \infty & 0 & \infty & 2 \\ \infty & \infty & 0 & 1 \\ \infty & \infty & \infty & 0 \end{bmatrix}$$

- Iteraciones:

- Vertice A como intermedio: No se producen cambios.
- Vertice B como intermedio: Actualiza $D[A][D] = 3$.
- Vertice C como intermedio: No se producen cambios.
- Vertice D como intermedio: No se producen cambios.

-
- Matriz final:

$$D = \begin{bmatrix} 0 & 1 & 5 & 3 \\ \infty & 0 & \infty & 2 \\ \infty & \infty & 0 & 1 \\ \infty & \infty & \infty & 0 \end{bmatrix}$$

Codigo en Java:

```
1  int [][] graph = {  
2      {0, 1, 5, INF},  
3      {INF, 0, INF, 2},  
4      {INF, INF, 0, 1},  
5      {INF, INF, INF, 0}  
6  };  
7  
8  FloydWarshall fw = new FloydWarshall();  
9  fw.floydWarshall(graph);
```